

# Snacki

APPLICATION CLIENT · PWA V1.1.0

## Guide de construction et de mise en ligne



Ce guide accompagne le dossier `snacki-pwa/`. Il explique ce que contient l'application, comment elle a été construite, étape par étape, et comment la mettre en ligne pour les clients de Snacki.

- Version : 1.1.0, 25 septembre 2026
- Périmètre : étape 1 sur 6 (application client : menu, panier, commande WhatsApp, français et arabe)
- Document lié : `Snacki_Livrable_1_Application_client.pdf` (explications détaillées, tests, sécurité)

### 1 Contenu du zip

```
Snacki_PWA_v1.1.0/
├─ snacki-pwa/           ← le dossier à mettre en ligne (et seulement lui)
│   ├─ index.html        la page : structure et en-tête PWA
│   ├─ styles.css        le design : couleurs, polices, mise en page, mode sombre
│   ├─ app.js            la logique : menu, panier, formulaire, message WhatsApp, illustrations
│   ├─ sw.js             le service worker : garde l'app en mémoire pour l'ouvrir hors connexion
│   ├─ manifest.webmanifest la carte d'identité de l'app : nom, icônes, couleurs
│   ├─ _headers          les règles de sécurité envoyées par Netlify
│   ├─ img/logo.jpg     le logo Snacki
│   ├─ icons/            les icônes de l'écran d'accueil (192, 512, iPhone, onglet)
│   └─ fonts/            les polices Tajawal et Baloo Bhaijaan 2 (arabe + latin)
├─ GUIDE-CONSTRUCTION.md ce guide (version texte)
├─ GUIDE-CONSTRUCTION.pdf ce guide (version PDF)
└─ Snacki_Livrable_1_Application_client.pdf
```

Le dossier `snacki-pwa` pèse environ 450 Ko. Il n'utilise aucun service externe : pas de serveur, pas de base de données, pas de Google Fonts. Tout est dans le dossier.

### 2 Les étapes de construction

Voici, dans l'ordre, comment l'application a été construite. Chaque étape s'appuie sur la précédente.

## Étape 1 : cadrer à partir des documents existants

Sources lues : le menu imprimé, le fichier `Suivi Dashboard 15.xlsx`, le rapport d'analyse globale et le document Architecture & Sécurité v2.

Ce qu'on en a tiré : - les 9 produits du menu, leurs noms arabes et leurs prix ; - les produits les plus vendus (salade de fruits, puis crêpe et jus d'avocat), d'où les badges « N°1 des ventes » et « Populaire » ; - les moyens de paiement du menu (espèces, Bankily, Sedad, Bimbank, Bamis Digital) et les numéros WhatsApp et d'appel ; - les points faibles de la v2 à ne pas reproduire (code PIN lisible dans le code, rien n'est enregistré).

Décisions validées ensuite avec l'équipe : prix en MRU, frais de livraison confirmés sur WhatsApp, descriptions courtes, caisse sur téléphone à l'étape 2.

## Étape 2 : décrire les données

Tout ce qui peut changer sans toucher à la logique est rangé dans trois blocs au début de `app.js` :

| Bloc     | Contenu                                                                          |
|----------|----------------------------------------------------------------------------------|
| CONFIG   | numéro WhatsApp, unité (MRU), quartiers de livraison, moyens de paiement         |
| PRODUCTS | les 9 produits : identifiant, catégorie, prix, noms et descriptions FR/AR, badge |
| T        | tous les textes de l'interface, en français et en arabe                          |

## Étape 3 : poser l'identité visuelle

L'app reprend la marque du menu imprimé : - couleurs : jaune Snacki `#FFC400`, brun presque noir `#1E1607`, fond crème `#FFFCF3`, vert feuille `#2F7D22`, rouge fraise `#D22F3D` ; - polices : Baloo Bhaijaan 2 pour les titres et les prix (ronde, gourmande, avec l'arabe), Tajawal pour le texte (très lisible en arabe et en français) ; - repères de la marque : les titres de section sur un bandeau noir incliné (comme « DÉLICES » et « JUS NATURELS » sur le menu), les prix dans une pastille jaune ; - un mode sombre automatique, qui suit le réglage du téléphone.

## Étape 4 : écrire l'application (état, puis rendu)

L'application est une page unique qui ne se recharge jamais. Tout ce qui change (langue, panier, étape de commande, formulaire) est rangé dans un objet `state`. Chaque action du client modifie `state`, puis la fonction `renderAll()` redessine l'écran à partir de `state`.

Le parcours tient en 3 écrans : menu, puis panier, puis informations, et enfin le message prêt à envoyer.

## Étape 5 : le français et l'arabe

- Un bouton FR / عربي bascule toute l'app ; en arabe, la page passe de droite à gauche (`dir="rtl"`).
- Le CSS utilise des propriétés « logiques » (début et fin de ligne) plutôt que gauche et droite, pour que tout s'inverse seul.
- Les nombres sont protégés par des caractères invisibles pour ne pas s'afficher à l'envers en arabe (« 000 1 » au lieu de « 1 000 »).

## Étape 6 : la commande WhatsApp

L'app rédige un message complet (produits, total, paiement, nom, téléphone, retrait ou livraison, repère) avec un numéro de commande `SNK-MMJJ-XXX`, puis ouvre `https://wa.me/22237939409?text=...`. Le client n'a plus qu'à appuyer sur Envoyer. Un bouton « Copier le message » sert de secours.

## Étape 7 : les visuels produits

Les photos prises au snack étaient floues et prises à la main. Elles sont remplacées par 9 illustrations dessinées par le code (format SVG) : le vrai gobelet Snacki à couvercle dôme, l'étiquette jaune, la couleur de chaque boisson et ses fruits. Elles sont nettes sur tous les écrans et ne pèsent presque rien. Une vraie photo peut les remplacer produit par produit (voir section 7).

## Étape 8 : tester et corriger

16 tests automatiques avec un vrai navigateur (Playwright), sur un écran de téléphone, en français, en arabe et en mode sombre. 6 bugs trouvés et corrigés : prix inversés en arabe, téléphone inversé dans le message arabe, compteur qui débordait de la carte, fenêtre cachée qui bloquait les boutons, notification qui cachait le bouton de validation, cercle décoratif qui masquait la fin du titre d'accueil. Le détail est dans le livrable PDF, section 9.

## Étape 9 : emballer en PWA

Une PWA (Progressive Web App) est un site qui s'installe comme une application. Pour passer de la démo à la PWA :

1. **Fichiers séparés.** La démo tenait en un seul fichier. La PWA sépare `index.html`, `styles.css` et `app.js`, ce qui permet une politique de sécurité stricte (aucun code écrit directement dans la page).
2. **Polices locales.** Les polices sont copiées dans `fonts/` : l'app ne dépend plus de Google et fonctionne hors connexion.
3. **Manifeste** (`manifest.webmanifest`) : nom « Snacki », couleur jaune, icônes, ouverture en plein écran sans barre de navigateur.
4. **Icônes** : le gobelet d'avocat Snacki sur fond jaune, en 192 et 512 px, plus une version « maskable » (Android découpe l'icône en cercle ou en carré arrondi) et une version iPhone.
5. **Service worker** (`sw.js`) : à la première visite, il met l'app en mémoire dans le téléphone. Ensuite, le menu s'ouvre même sans réseau. La page est toujours redemandée au serveur en premier, pour que les nouveaux prix arrivent dès qu'il y a du réseau.
6. **En-têtes de sécurité** (`_headers`) : lus automatiquement par Netlify (voir section 8).

Vérifications faites sur la PWA servie en HTTP : service worker installé, 23 fichiers en cache, polices chargées, commande complète, ouverture hors connexion, aucune violation de la politique de sécurité.

## 3 Tester sur votre ordinateur

Ouvrir `index.html` en double-cliquant fonctionne pour voir l'app, mais sans le mode hors connexion : le service worker exige une adresse `http://localhost` ou `https://`.

Pour un test complet, avec Python installé :

```
cd snacki-pwa
python3 -m http.server 8000
```

Puis ouvrir `http://localhost:8000` dans Chrome. Pour voir la PWA : clic droit → Inspecter → onglet **Application** → Manifest et Service workers.

## 4 Mettre en ligne sur Netlify (gratuit)

Netlify héberge gratuitement les sites comme celui-ci, avec HTTPS automatique (obligatoire pour une PWA).

1. Décompresser le zip sur l'ordinateur.
2. Créer un compte sur **app.netlify.com** (avec un e-mail ou un compte GitHub).
3. Dans le tableau de bord : **Add new site** (ou **Add new project**, selon la version de Netlify) → **Deploy manually**.
4. Glisser-déposer le dossier **snacki-pwa** (le dossier, pas le zip entier) dans la zone prévue.
5. Netlify donne une adresse du type `https://nom-au-hasard-123.netlify.app`. L'ouvrir sur un téléphone pour vérifier.
6. Changer le nom : **Site configuration** → **Change site name** (ou **Project configuration** → **Change project name**) → par exemple `snacki` ou `snacki-ndb`. L'adresse devient `https://snacki-ndb.netlify.app` (si le nom est libre).
7. Vérifier la sécurité : ouvrir `https://securityheaders.com`, coller l'adresse du site, lancer l'analyse. Les en-têtes du fichier `_headers` doivent apparaître.

**Mettre à jour le site** : Netlify → **Deploys** → glisser à nouveau le dossier `snacki-pwa` modifié. L'ancienne version reste disponible dans l'historique et peut être restaurée en un clic.

**Nom de domaine (facultatif)** : on peut acheter un nom de domaine (par exemple `snacki-ndb.com`) et le relier au site dans **Domain management**. C'est payant (quelques euros par an selon l'extension) ; l'adresse `.netlify.app` suffit pour démarrer.

**Autre hébergeur** : GitHub Pages ou tout hébergement HTTPS fonctionne aussi. Le fichier `_headers` est alors ignoré, mais la politique de sécurité principale reste active, car elle est aussi écrite dans `index.html`.

## 5 Installer l'app sur un téléphone

- **Android (Chrome)** : ouvrir le site → menu ⋮ → **Installer l'application** (ou **Ajouter à l'écran d'accueil**).
- **iPhone (Safari)** : ouvrir le site → bouton **Partager** → **Sur l'écran d'accueil**.

L'icône Snacki apparaît sur l'écran d'accueil ; l'app s'ouvre en plein écran, comme une application.

## 6 Faire connaître l'app aux clients

- Mettre le lien dans le statut WhatsApp du snack et dans la bio TikTok `@snackifood`.
- Imprimer un QR code du lien sur le menu, sur les gobelets ou sur la vitrine. N'importe quel générateur de QR code gratuit convient ; tester le QR code avec deux téléphones avant d'imprimer.
- Répondre aux clients qui écrivent sur WhatsApp avec le lien, pour qu'ils prennent l'habitude de commander par l'app.

## 7 Modifier l'application

Tous les changements courants se font au début de `app.js`, avec un éditeur de texte (par exemple VS Code ou Notepad++). Après chaque modification, suivre la section « Publier une modification » plus bas.

## Changer un prix

Chercher le produit dans `PRODUCTS` et changer `price` (en MRU) :

```
{id:'crepe', cat:'delices', price:120, badge:'pop',  
  fr:'Crêpe', ar:'الكريب',  
  dfr:'Chocolat et banane', dar:'شوكولاتة وموز'},
```

## Ajouter un produit

Copier une ligne existante, changer l'identifiant (`id`, sans espace ni accent), la catégorie (`'jus'` ou `'delices'`), le prix, les noms et les descriptions. Pour le badge, mettre `badge:'top'`, `badge:'pop'` ou retirer le champ. Un produit sans illustration dessinée affiche un carré de couleur vide : dans ce cas, lui ajouter une photo (paragraphe suivant).

## Utiliser une vraie photo pour un produit

1. Mettre la photo dans `img/`, carrée, environ 600 × 600 px, en JPG (par exemple `img/jus-avocat.jpg`).
2. Ajouter `photo:'img/jus-avocat.jpg'` dans le produit.
3. Ajouter `'img/jus-avocat.jpg'` dans la liste `ASSETS` de `sw.js`, pour qu'elle soit disponible hors connexion.

Conseils de prise de vue : fond uni et propre, lumière du jour près d'une fenêtre, même angle et même distance pour tous les produits, sans main, sans filigrane, gobelet bien rempli.

## Changer le numéro WhatsApp

Dans `CONFIG`, champ `whatsapp`, au format international sans « + » : `222` suivi des 8 chiffres.

## Changer un texte de l'interface

Dans `T`, modifier la phrase dans `fr` et dans `ar`.

## Publier une modification

1. Ouvrir `sw.js` et changer le numéro de version : `const CACHE = 'snacki-v1.1.0';` → `const CACHE = 'snacki-v1.1.1';`
2. Redéployer le dossier sur Netlify (section 4).

Pourquoi : les téléphones gardent l'app en mémoire. Changer le nom du cache leur indique qu'il y a une nouvelle version à télécharger. Sans ce changement, certains clients verraient encore les anciens prix pendant un moment.

## 8 Sécurité intégrée

| Protection                                              | Où                                               | Effet                                                                               |
|---------------------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------|
| Politique de sécurité du contenu (CSP) stricte          | <code>index.html</code> et <code>_headers</code> | La page n'exécute que ses propres fichiers ; un code injecté ne peut pas s'exécuter |
| Textes du client insérés comme texte, jamais comme HTML | <code>app.js</code>                              | Protection contre l'injection de code (XSS)                                         |

| Protection                                                                              | Où                    | Effet                                                                                               |
|-----------------------------------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------|
| Contrôle des champs                                                                     | <code>app.js</code>   | Téléphone mauritanien à 8 chiffres (2, 3 ou 4), repère obligatoire en livraison, longueurs limitées |
| Nettoyage du panier gardé dans le téléphone                                             | <code>app.js</code>   | Un panier trafiqué (quantités folles, produit inventé) est ignoré                                   |
| Retours à la ligne supprimés des champs libres                                          | <code>app.js</code>   | Impossible d'insérer une fausse ligne « Total » dans le message                                     |
| Anti-intégration ( <code>frame-ancestors 'none'</code> , <code>X-Frame-Options</code> ) | <code>_headers</code> | Le site ne peut pas être piégé dans une autre page                                                  |
| HTTPS forcé ( <code>Strict-Transport-Security</code> )                                  | <code>_headers</code> | Les échanges ne peuvent pas être lus ou modifiés en route                                           |
| Permissions coupées (caméra, micro, position, paiement)                                 | <code>_headers</code> | Le site ne peut pas les demander                                                                    |
| Aucun secret dans le code                                                               | toute l'app           | Il n'y a rien à voler : pas de mot de passe, pas de code PIN                                        |

Limite connue : les commandes arrivent sur WhatsApp et l'équipe vérifie le total avec le menu. L'enregistrement sécurisé des commandes par un serveur arrive à l'étape 3.

## 9 Dépannage

| Problème                                              | Solution                                                                                                                                                   |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Les modifications n'apparaissent pas sur un téléphone | Vérifier que la version dans <code>sw.js</code> a changé ; fermer complètement l'app et la rouvrir                                                         |
| « Installer l'application » n'apparaît pas            | Le site doit être en <code>https://</code> ; ouvrir le site dans Chrome (Android) ou Safari (iPhone), pas dans le navigateur interne de WhatsApp ou TikTok |
| WhatsApp ne s'ouvre pas                               | Vérifier que WhatsApp est installé ; sinon, le client utilise « Copier le message » et l'envoi au 37 93 94 09                                              |
| L'icône n'a pas changé                                | Supprimer l'app de l'écran d'accueil et la réinstaller                                                                                                     |

## 10 Et ensuite

L'étape 2 construit la **caisse** de la gestionnaire, sur téléphone : saisie des commandes au comptoir en quelques taps, commandes du jour avec statut (reçue, en préparation, prête, livrée) et total du jour en direct. Elle remplacera la saisie en texte libre de l'onglet « Ventes » de l'Excel.